

Task 1: Generate a Random Maze

Submission report | Computer Graphics and Multimedia

Objective

Generate and display a random maze in Unity that satisfies the three requirements from the assignment: connected corridors, editable dimensions, and loops.

Implementation

Maze.cs initializes a wall-filled grid and uses randomized depth-first search to visit each room and carve passages. This creates a connected spanning tree. AddLoops() opens additional internal walls with a 10% chance, creating cycles. Edit ROWS and COLS near the top of Maze.cs; defaults are 11 rows and 21 columns. Press Space during runtime to destroy and regenerate the maze.

Evidence included

Unity Inspector, default maze, loop example, two dimension configurations and generated results, plus code screenshots for Carve() and AddLoops().

Dimension tests: 21 x 41 and 42 x 82 (rows x columns).

Upload this PDF to the virtual campus activity and present the work package to the professor, who may ask questions about the implementation.

Figure 1: Unity setup

Maze component and prefab assignments in the Inspector

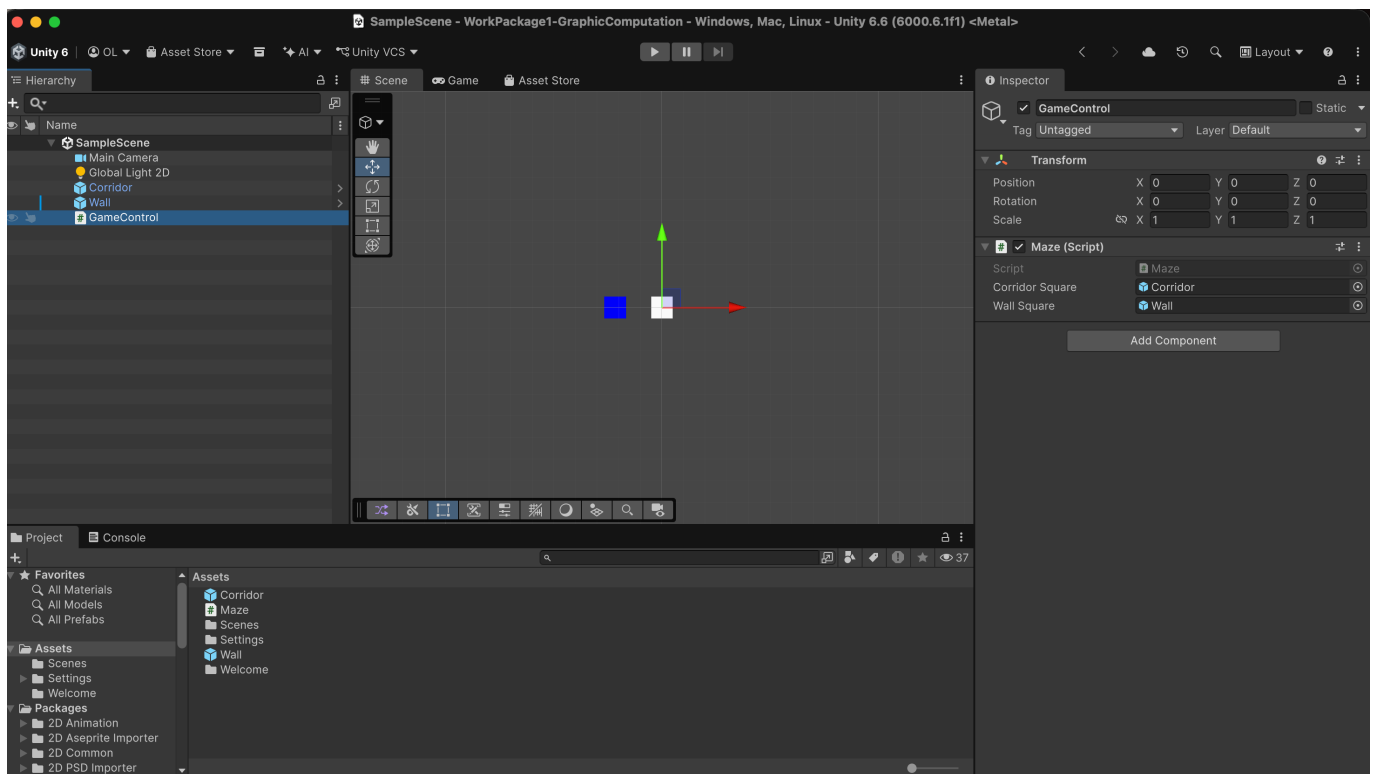


Figure 2: Generated maze

Default size: 11 rows x 21 columns

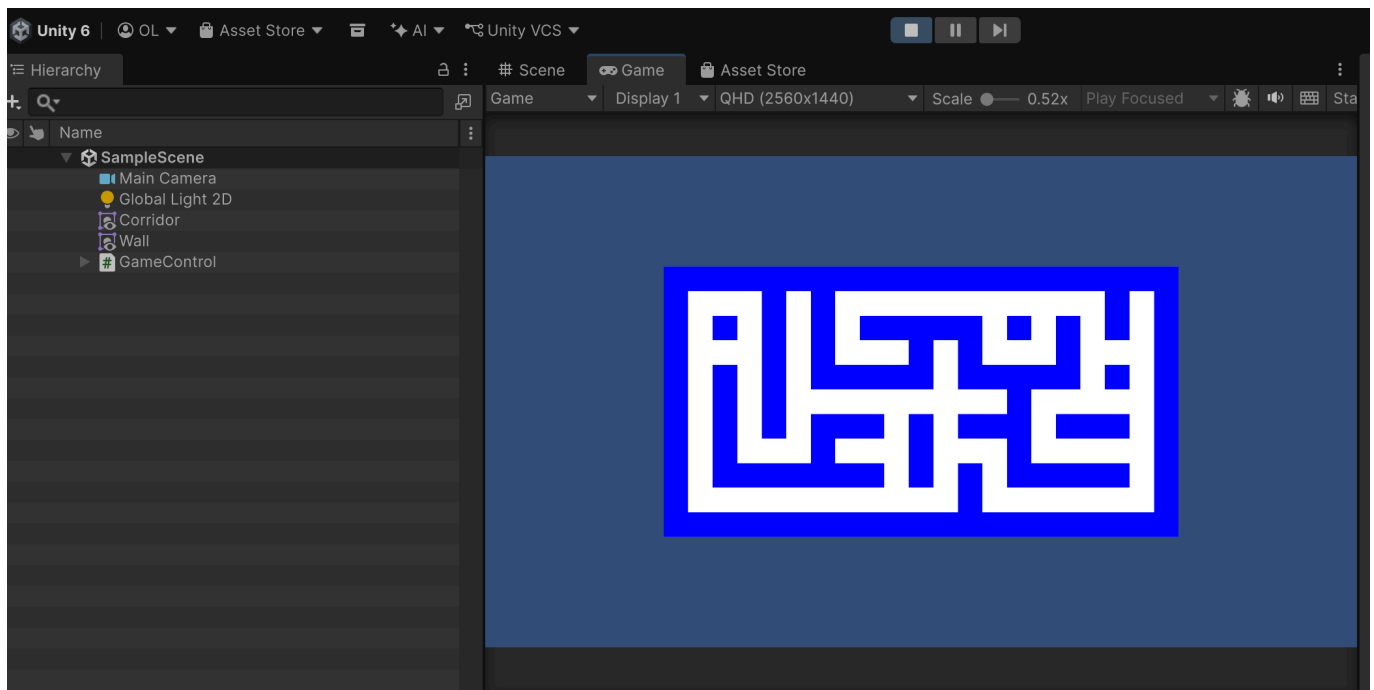


Figure 3: Loop evidence

Generated maze view with alternative routes highlighted

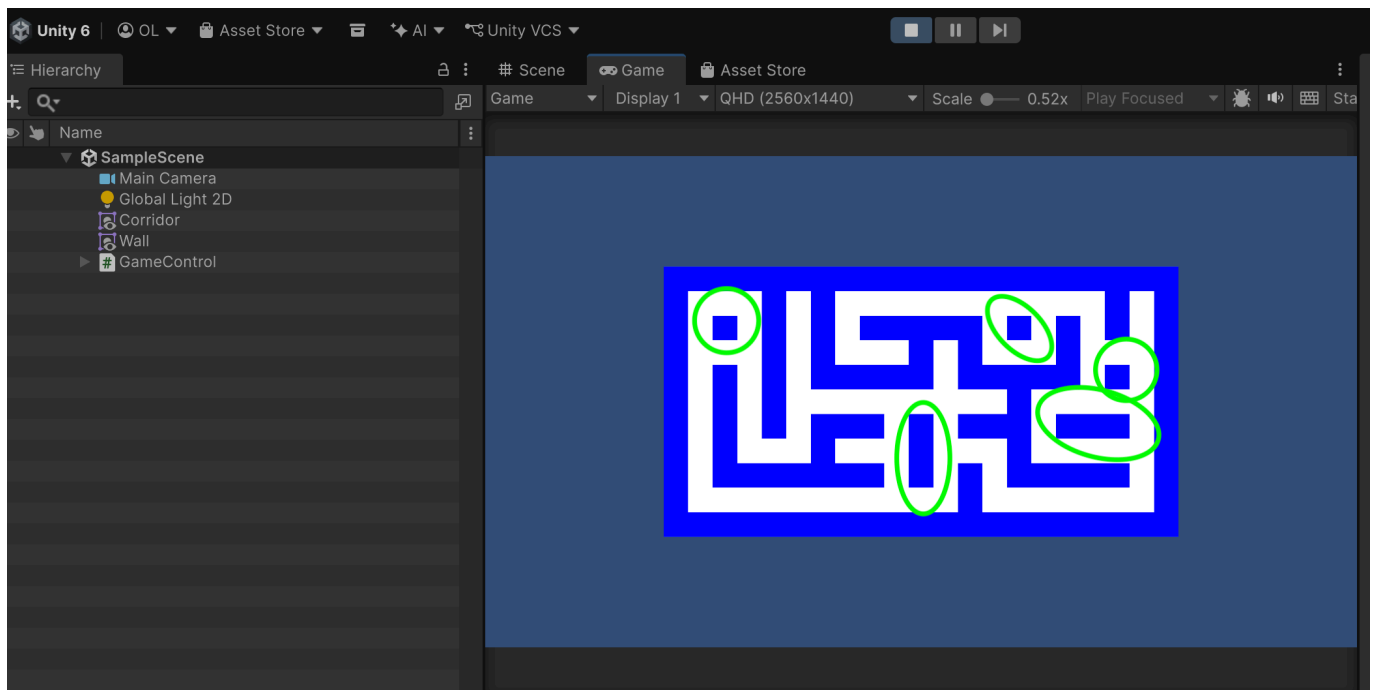


Figure 4: Dimension changes

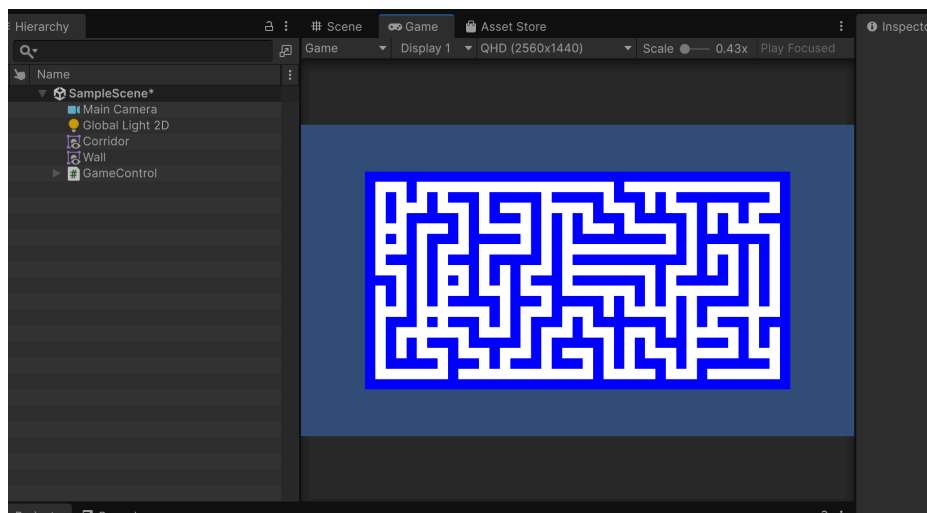
Configurations and generated output; dimensions shown as rows x columns

21 x 41: configuration

```
// ROWS and COLS must be ODD numbers (needed for the room/w
// To change the maze size, just change these two values.
int ROWS = 21;
int COLS = 41;

int[,] maze;
```

21 x 41: generated maze



42 x 82: configuration

```
// ROWS and COLS must be ODD numbers (n
// To change the maze size, just change
int ROWS = 41;
int COLS = 82;
```

Figure 4: Dimension changes (continued)

Generated output at 42 rows x 82 columns

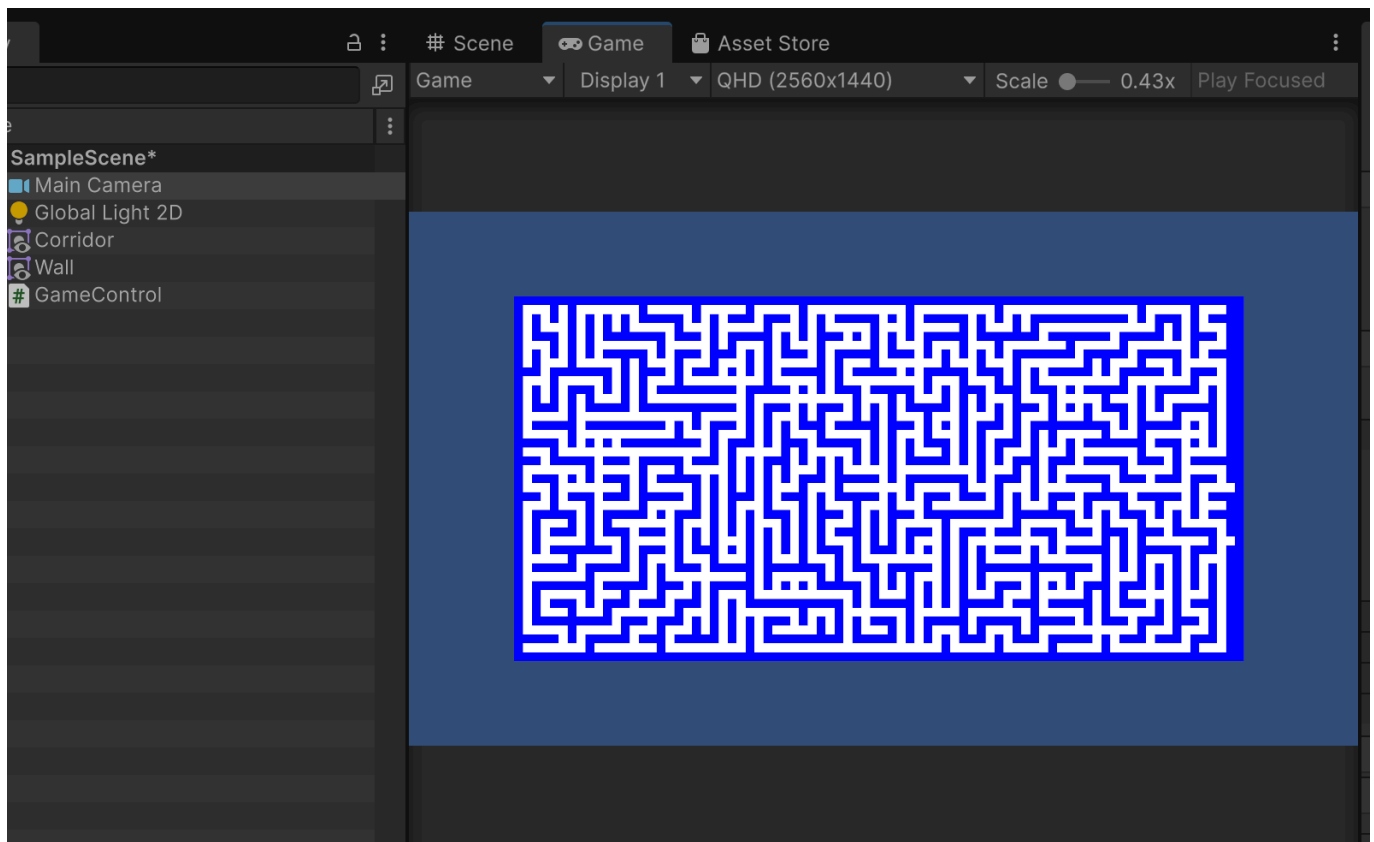


Figure 5: Maze generation code

Recursive carving

```
void Carve(int i, int j)
{
    maze[i, j] = 0;
    visited[i, j] = true;

    // Visit the 4 neighbouring rooms (2 cells away) in random order.
    int[] order = { 0, 1, 2, 3 };
    Shuffle(order);

    for (int k = 0; k < 4; k++)
    {
        int ni = i, nj = j;
        switch (order[k])
        {
            case 0: ni = i - 2; break; // up
            case 1: ni = i + 2; break; // down
            case 2: nj = j - 2; break; // left
            case 3: nj = j + 2; break; // right
        }

        if (ni <= 0 || ni >= ROWS - 1 || nj <= 0 || nj >= COLS - 1)
            continue;

        if (visited[ni, nj])
            continue;

        // Knock down the wall between (i,j) and (ni,nj).
        maze[(i + ni) / 2, (j + nj) / 2] = 0;

        Carve(ni, nj);
    }
}
```

Figure 5: Maze generation code (continued)

AddLoops() post-processing

```
void AddLoops()
{
    // Go through every wall that sits between two rooms and, with a
    // small chance, knock it down too. The maze is already fully
    // connected at this point, so this only ever ADDS extra paths
    // (loops) between existing corridor cells - it can never
    // disconnect anything.
    float loopChance = 0.1f;

    for (int i = 1; i < ROWS - 1; i++)
    {
        for (int j = 1; j < COLS - 1; j++)
        {
            bool isHorizontalWall = (i % 2 == 1) && (j % 2 == 0);
            bool isVerticalWall = (i % 2 == 0) && (j % 2 == 1);

            if ((isHorizontalWall || isVerticalWall) && maze[i, j] == 1)
            {
                if (Random.value < loopChance)
                {
                    maze[i, j] = 0;
                }
            }
        }
    }
}
```